SPEC

SPEC.md — HelixKnowledge Skill Graph System

Revision: 1 **Last modified:** 2026-07-17T23:11:30Z

1. Overview

Revision: 1 **Last modified:** 2026-07-17T23:11:30Z

A self-growing Knowledge Skill Graph system for AI CLI agents. Each Skill is a versioned unit of knowledge for a specific technology, with recursive dependencies forming a DAG. The system auto-detects gaps, validates knowledge through multi-layer defense, and learns from real codebases.

Key corrections from research: - ACP uses JSON-RPC over stdio (NOT gRPC) - TOML for config/skill definitions; JSON for API wire format - HTTP/2 default for local; HTTP/3 via Caddy for remote - 3-model jury optimal for validation - 768d embeddings sweet spot; support pluggable providers

---

2. Technology Stack

Component	Choice	Version
Language	Go	1.22+
API Framework	Gin	v1.11.0+
HTTP/3	quic-go + Caddy (proxy)	Latest
Compression	andybalholm/brotli	Latest

Component	Choice	Version
Config Format	TOML (BurntSushi/toml)	v1.6.0
API Format	JSON primary, TOML optional	-
Database	PostgreSQL	16+
Vector Ext	pgvector	0.8.0+
Embedding	OpenAI text-embedding-3-small (default), pluggable	-
Code Parsing	tree-sitter (official Go bindings)	Latest
MCP Server	mcp-go (mark3labs)	v0.56.0+
CLI Framework	Cobra	Latest
TUI Framework	Bubble Tea	Latest
Deployment	Docker Compose / Podman Compose	-

---

3. Project Structure

```
skill-system/
├── cmd/
│   ├── server/           # REST API + MCP server (main entry: main.go)
│   ├── worker/           # Background jobs (autoexpand, codeanalysis)
│   ├── cli/              # Cobra CLI tool
│   └── tui/              # Bubble Tea TUI
├── internal/
│   ├── config/           # TOML config loading, env vars
│   ├── db/               # PostgreSQL + pgvector client, migrations
│   ├── models/           # Shared data structures
│   ├── skill/            # Skill CRUD, dependency graph, recursive queries
│   ├── registry/         # Central registrar, health monitoring
│   ├── autoexpand/       # Auto-growth pipeline
│   ├── codeanalysis/     # tree-sitter integration, pattern extraction
│   ├── validation/       # Multi-model jury, sandbox, source verification
│   ├── mcp/              # MCP server (mcp-go), tool definitions
```

```
│   ├── api/             # Gin handlers, middleware, content negotiation
│   └── worker/          # Background job runner
├── migrations/          # SQL migration files
├── scripts/             # Lifecycle bash scripts
├── config/
│   └── config.toml      # Default configuration template
├── docker-compose.yml
├── Dockerfile
├── go.mod
├── go.sum
└── README.md
```

---

4. Data Models

4.1 Core Types (internal/models/)

```go
// Skill represents a single knowledge unit
type Skill struct {
    ID          uuid.UUID       `json:"id" db:"id"`
    Name        string          `json:"name" db:"name"`             // e.g., "android.aosp.build-system"
    Version     string          `json:"version" db:"version"`       // SemVer
    Title       string          `json:"title" db:"title"`
    Description string          `json:"description" db:"description"`
    Content     string          `json:"content" db:"content"`       // Full Markdown
    Metadata    json.RawMessage `json:"metadata" db:"metadata"`     // tags, domain, complexity
    Embedding   pgvector.Vector `json:"-" db:"embedding"`           // 768d or 1536d
    Status      SkillStatus     `json:"status" db:"status"`         // draft | validated | active | deprecated
    Kind        SkillKind       `json:"kind" db:"kind"`             // NEW (R16) — atomic (default) | composite | umbrella
    CreatedAt   time.Time       `json:"created_at" db:"created_at"`
    UpdatedAt   time.Time       `json:"updated_at" db:"updated_at"`
}

type SkillStatus string
```

```go
const (
    SkillStatusDraft      SkillStatus = "draft"
    SkillStatusValidated  SkillStatus = "validated"
    SkillStatusActive     SkillStatus = "active"
    SkillStatusDeprecated SkillStatus = "deprecated"
)

// DependencyType defines how skills relate
type DependencyType string
const (
    DepTypeRequires   DependencyType = "requires"   // existing — hard closure
    DepTypeExtends    DependencyType = "extends"    // existing — hard closure
    DepTypeRecommends DependencyType = "recommends" // existing — advisory

    // R16 granularity/composition additions (research/skill_granularity_and_composition.md §4.1).
    DepTypeComposes    DependencyType = "composes"       // NEW — hard closure, whole->part aggregation
    DepTypeRelatedTo   DependencyType = "related_to"     // NEW — advisory, symmetric "see also"
    DepTypeAlternative DependencyType = "alternative_to" // NEW — advisory, symmetric substitute
)

// HardClosureTypes is the set the "everything needed for X" resolver walks.
var HardClosureTypes = []DependencyType{DepTypeRequires, DepTypeComposes, DepTypeExtends}

// SkillKind classifies a skill on the aggregation axis (orthogonal to
// Metadata.Complexity). See research/skill_granularity_and_composition.md §3.1.
type SkillKind string
const (
    SkillKindAtomic    SkillKind = "atomic"    // indivisible building block (default)
    SkillKindComposite SkillKind = "composite" // mid-level aggregator
    SkillKindUmbrella  SkillKind = "umbrella"  // technology/stack root; wizard entry point
)

// SkillDependency represents a directed edge in the skill DAG
type SkillDependency struct {
    SkillID      uuid.UUID      `json:"skill_id" db:"skill_id"`
```

```go
    DependsOn    uuid.UUID      `json:"depends_on" db:"depends_on"`
    RelationType DependencyType `json:"relation_type" db:"relation_type"`
    Optional     bool           `json:"optional" db:"optional"`           // NEW (R16) — default false
    SortOrder    *int           `json:"sort_order,omitempty" db:"sort_order"` // NEW (R16) — component ordering; nil = unordered
}

// Resource is an external reference (URL to docs, articles, code)
type Resource struct {
    ID            uuid.UUID  `json:"id" db:"id"`
    SkillID       uuid.UUID  `json:"skill_id" db:"skill_id"`
    URL           string     `json:"url" db:"url"`
    Title         string     `json:"title" db:"title"`
    ResourceType  string     `json:"resource_type" db:"resource_type"` // official-doc, article, code, video
    FetchedHash   string     `json:"fetched_hash" db:"fetched_hash"`   // SHA256
    ContentCached string     `json:"content_cached" db:"content_cached"`
    LastValidated *time.Time `json:"last_validated" db:"last_validated"`
}

// Evidence is a learned experience from a real codebase
type Evidence struct {
    ID            uuid.UUID      `json:"id" db:"id"`
    SkillID       uuid.UUID      `json:"skill_id" db:"skill_id"`
    SourceProject string         `json:"source_project" db:"source_project"`
    SourceFile    string         `json:"source_file" db:"source_file"`
    CodeSnippet   string         `json:"code_snippet" db:"code_snippet"`
    Pattern       string         `json:"pattern" db:"pattern"`
    Language      string         `json:"language" db:"language"`
    Validated     bool           `json:"validated" db:"validated"`
    Embedding     pgvector.Vector `json:"-" db:"embedding"`
    CreatedAt     time.Time      `json:"created_at" db:"created_at"`
}

// SkillRegistry tracks health and completeness
type SkillRegistryEntry struct {
    SkillID    uuid.UUID `json:"skill_id" db:"skill_id"`
    SkillName  string    `json:"skill_name" db:"skill_name"`
```

```go
    MissingDeps []string  `json:"missing_deps" db:"missing_deps"`
    Stale       bool      `json:"stale" db:"stale"`
    LastReview  *time.Time `json:"last_review" db:"last_review"`
    AutoExpand  bool      `json:"auto_expand" db:"auto_expand"`
    Coverage    float64   `json:"coverage" db:"coverage"` //
        0.0-1.0
}

// AuditLogEntry tracks all system events
type AuditLogEntry struct {
    Timestamp time.Time       `json:"ts" db:"ts"`
    Event     string          `json:"event" db:"event"`
    SkillID   *uuid.UUID      `json:"skill_id,omitempty"
        db:"skill_id"`
    Details   json.RawMessage `json:"details" db:"details"`
}
```

4.2 TOML Skill Format (for API import/export and human editing)

```toml
[skill]
name = "android.aosp.build-system"
version = "0.1.0"
title = "AOSP Build System (Soong, Make, Bazel)"
description = "Complete reference for Android build, Soong
        blueprints, Android.bp, etc."
kind = "composite"
        # NEW (R16) — atomic (default, may be omitted) | composite
        | umbrella
content = """
# AOSP Build System

## Overview
The Android build system uses Soong (Android.bp), Make
        (Android.mk), and migrating to Bazel.

## Soong Blueprints
...full markdown content...
"""

[skill.metadata]
tags = ["android", "build", "soong", "bazel"]
domain = "android"
complexity = "intermediate"

[skill.dependencies]
```

```toml
requires       = ["linux.kernel-modules", "python.basics", "make.basics"]
extends        = ["android.general"]
recommends     = ["bazel.advanced"]
composes       = ["android.build.soong", "android.build.kati"]  # NEW (R16) — component leaves this node aggregates
related_to     = []                                             # NEW (R16) — symmetric "see also"
alternative_to = []                                             # NEW (R16) — symmetric substitute
# depends_on / prerequisite / part_of are also ACCEPTED here and normalized
# per §4.1 of research/skill_granularity_and_composition.md

# OPTIONAL ergonomic authoring form for composite/umbrella skills that need
# per-component ordering/optionality. Each entry materializes as one
# `composes` edge.
[[skill.components]]
name  = "android.build.soong"
order = 1
optional = false

[[skill.resources]]
url = "https://source.android.com/docs/setup/build/building"
title = "Official Android Build Documentation"
resource_type = "official-doc"

[[skill.resources]]
url = "https://android.googlesource.com/platform/build/soong/+/refs/heads/main/README.md"
title = "Soong README"
resource_type = "code"
```

---

5. Database Schema (migrations/001_initial.up.sql + migrations/002_granularity.up.sql)

Current (post-002_granularity) cumulative shape — see `research/p1t1_granularity_schema_migration.md` §1 for the additive `ALTER` migration that took 001's schema to this shape, and `research/skill_granularity_and_composition.md` §5.3 for the model it implements.

```sql
-- Enable required extensions
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- Skills table
CREATE TABLE skills (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name            TEXT NOT NULL UNIQUE,
    version         TEXT NOT NULL DEFAULT '0.1.0',
    title           TEXT NOT NULL,
    description     TEXT,
    content         TEXT NOT NULL,
    metadata        JSONB NOT NULL DEFAULT '{}',
    embedding       vector(768),  -- 768d default (sweet spot per
        research)
    created_at      TIMESTAMPTZ DEFAULT NOW(),
    updated_at      TIMESTAMPTZ DEFAULT NOW(),
    status          TEXT DEFAULT 'draft' CHECK (status IN ('draft',
        'validated', 'active', 'deprecated')),
    kind            TEXT NOT NULL DEFAULT 'atomic' CHECK (kind IN
        ('atomic', 'composite', 'umbrella'))  -- NEW (R16,
        002_granularity)
);
CREATE INDEX idx_skills_name ON skills(name);
CREATE INDEX idx_skills_status ON skills(status);
CREATE INDEX idx_skills_metadata ON skills USING GIN(metadata);
CREATE INDEX idx_skills_kind ON skills(kind);  -- NEW (R16,
        002_granularity)

-- Skill dependencies (DAG edges)
CREATE TABLE skill_dependencies (
    skill_id        UUID REFERENCES skills(id) ON DELETE CASCADE,
    depends_on      UUID REFERENCES skills(id) ON DELETE CASCADE,
    relation_type TEXT NOT NULL DEFAULT 'requires' CHECK
        (relation_type IN (
        'requires', 'extends', 'recommends',        -- existing
        (001)
        'composes', 'related_to', 'alternative_to'  -- NEW (R16,
        002_granularity)
    )),
    optional        BOOLEAN NOT NULL DEFAULT FALSE,    -- NEW (R16,
        002_granularity)
    sort_order      INT,
        -- NEW (R16, 002_granularity) — nullable; NULL = unordered
    PRIMARY KEY (skill_id, depends_on, relation_type)  -- widened
        (R16, 002_granularity) from (skill_id, depends_on)
);
CREATE INDEX idx_deps_skill ON skill_dependencies(skill_id);
```

```sql
CREATE INDEX idx_deps_depends_on ON
        skill_dependencies(depends_on);

-- Resources
CREATE TABLE resources (
    id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    skill_id       UUID REFERENCES skills(id) ON DELETE CASCADE,
    url            TEXT NOT NULL,
    title          TEXT,
    resource_type  TEXT DEFAULT 'article' CHECK (resource_type IN
        ('official-doc', 'article', 'code', 'video', 'tutorial')),
    fetched_hash   TEXT,
    content_cached TEXT,
    last_validated TIMESTAMPTZ,
    created_at     TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_resources_skill ON resources(skill_id);

-- Evidence (learned from codebases)
CREATE TABLE evidences (
    id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    skill_id       UUID REFERENCES skills(id) ON DELETE CASCADE,
    source_project TEXT NOT NULL,
    source_file    TEXT,
    code_snippet   TEXT,
    pattern        TEXT,
    language       TEXT,
    validated      BOOLEAN DEFAULT FALSE,
    embedding      vector(768),
    created_at     TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_evidences_skill ON evidences(skill_id);
CREATE INDEX idx_evidences_project ON evidences(source_project);

-- Skill registry (health tracking)
CREATE TABLE skill_registry (
    skill_id     UUID PRIMARY KEY REFERENCES skills(id),
    skill_name   TEXT NOT NULL,
    missing_deps TEXT[] DEFAULT '{}',
    stale        BOOLEAN DEFAULT FALSE,
    last_review  TIMESTAMPTZ,
    auto_expand  BOOLEAN DEFAULT TRUE,
    coverage     FLOAT DEFAULT 0.0
);
CREATE INDEX idx_registry_stale ON skill_registry(stale);
```

```sql
-- Audit log
CREATE TABLE audit_log (
    ts          TIMESTAMPTZ DEFAULT NOW(),
    event       TEXT NOT NULL,
    skill_id    UUID REFERENCES skills(id),
    details     JSONB DEFAULT '{}'
);
CREATE INDEX idx_audit_ts ON audit_log(ts);
CREATE INDEX idx_audit_event ON audit_log(event);

-- HNSW index for skill embeddings (pgvector)
CREATE INDEX idx_skills_embedding ON skills USING hnsw(embedding
        vector_cosine_ops)
    WITH (m = 32, ef_construction = 128);

-- HNSW index for evidence embeddings
CREATE INDEX idx_evidences_embedding ON evidences USING
        hnsw(embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- Trigger: update updated_at on skills
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ language 'plpgsql';

CREATE TRIGGER update_skills_updated_at BEFORE UPDATE ON skills
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```

---

6. API Specification

All endpoints under `/api/v1`. Auth via `X-API-Key` header.

6.1 Skills

- `GET /skills` — List skills (query: `status`, `domain`, `tag`, `search`, `limit`, `offset`)
- `GET /skills/:name` — Get skill by name (query: `recursive=true` for full tree)

- `POST /skills` — Create skill (body: Skill JSON or TOML)
- `PUT /skills/:name` — Update skill
- `DELETE /skills/:name` — Delete skill (cascades dependencies)
- `GET /skills/:name/tree` — Get dependency tree (query: `depth`, `format=json|toml`)
- `POST /skills/import` — Bulk import from TOML
- `GET /skills/:name/export` — Export skill + dependencies as TOML

6.2 Search

- `POST /search` — Vector + keyword hybrid search (body: `{ "query": "...", "limit": 5 }`)
- `POST /search/similar` — Find skills similar to given content

6.3 Registry

- `GET /registry` — Full registry with health status
- `GET /registry/missing` — List all missing dependencies
- `GET /registry/stale` — List stale skills
- `POST /registry/review/:name` — Trigger manual review
- `GET /registry/coverage` — Coverage report

6.4 Auto-Expand

- `POST /expand/:name` — Trigger auto-expansion for a skill
- `GET /expand/status/:id` — Check expansion job status
- `POST /expand/gap-report` — Generate gap analysis report

6.5 Learning

- `POST /learn` — Submit project path for analysis (body: `{ "project_path": "...", "languages": ["java", "kotlin"] }`)
- `GET /learn/status/:id` — Check analysis job status
- `GET /evidences/:skill_name` — Get evidence for a skill

6.6 System

- `GET /health` — Health check
- `GET /metrics` — Prometheus metrics
- `GET /version` — Version info

6.7 Content Negotiation

- Default: `application/json`
- TOML support: `Accept: application/toml` → TOML response
- Import accepts both JSON and TOML (auto-detected or via Content-Type)

---

7. MCP Server Tools

The MCP server exposes these tools via stdio transport:

Tool	Description
`skill_search`	Vector/hybrid search for skills
`skill_get`	Retrieve full skill by name
`skill_tree`	Get dependency tree
`skill_create`	Create a new skill
`learn_from_project`	Submit a project for analysis
`missing_skills`	List gaps in the knowledge graph
`get_coverage`	Get coverage report for a domain

---

8. Configuration (config.toml)

```toml [server] host = "0.0.0.0" http_port = 8080 http3_port = 8443 enable_http3 = false # default HTTP/2, enable HTTP/3 for remote enable_brotli = true tls_cert = "" tls_key = ""

[database] host = "db" port = 5432 database = "skilldb" user = "skill" password = "secret" ssl_mode = "disable" max_connections = 25

[embedding] provider = "openai" # openai | local | anthropic dimensions = 768 model = "text-embedding-3-small" api_key = "${OPENAI_API_KEY}" local_endpoint = "" # for local models (ollama, etc.)

[validation] enabled = true sandbox_type = "wasm" # wasm | gvisor | docker jury_size = 3 approval_threshold = 2 # min approvals from jury auto_approve_evidence = false require_human_review = true

[autoexpand] enabled = true max_depth = 5 max_new_skills_per_run = 10 llm_provider = "openai" llm_model = "gpt-4o-mini"
```

[codeanalysis] enabled = true languages = [“java”, “kotlin”, “c”, “cpp”, “python”, “go”] max_file_size_kb = 500 exclude_patterns = [“vendor/”, “node_modules/”, “.git/”, “build/”]

[mcp] enabled = true transport = “stdio” # stdio | http

[registry] review_interval_hours = 24 coverage_threshold = 0.8

[logging] level = “info” # debug | info | warn | error format = “json” # json | text